

Fourier Motzkin Elimination

傅里叶-莫茨金消元法

引入

傅里叶——莫茨金消元法（原名 Fourier–Motzkin Elimination，简称 **FME** 算法）是一种用于从线性不等式中消除变量的数学方法。

它的命名源自于在 1827 年和 1936 年独立发现该算法的 Joseph Fourier 和 Theodore Motzkin 的姓氏。

过程

从线性不等式中消除一组变量，是指通过将关系式中的若干个元素有限次地变换，消去其中的某些元素，从而解决问题的一种方法。

如果线性不等式中的所有变量都被消除，那么我们会得到一个常不等式。因为当且仅当原不等式有解时，消元后的不等式才为真，消除所有变量可用于检测不等式系统是否有解。

考虑一个含 n 个不等式的系统，有从 1 到 n 的 n 个变量，其中 i 为要消除的变量。根据 a_i 系数的符号（正、负或空）， S 中的线性不等式可以分为三类：

1. 形式为 $a_i x_i \leq b$ 的不等式，对于范围从 1 到 $i-1$ （为这种不等式的数量）的 x_j ，用 S_1 表示；
2. 形式为 $a_i x_i \geq b$ 的不等式，对于范围从 1 到 $i-1$ （为这种不等式的数量）的 x_j ，用 S_2 表示；
3. 不包含 x_i 的不等式，设它们构成的不等式组为 S_3 。

因此原系统等价于

消元包括产生一个等价于 $S_1 \cup S_2 \cup S_3$ 的系统。显然，这个公式等价于

不等式

等价于对于 $i \in \{1, \dots, n\}$ 且 $a_i \neq 0$ ，所有 $S_1 \cup S_2$ 个不等式 构成的不等式组。

因此，我们将原系统 S 转换为另一个消掉 x_i 的系统，这个系统有 $|S_1 \cup S_2|$ 个不等式。特别地，如果 $a_i = 0$ ，那么新系统不等式的个数为 $|S|$ 。

例题

考虑以下不等式系统：

为了消除，我们可以根据 改写不等式：

这样我们得到两个 不等式和两个 不等式；如果每个 不等式的右侧至少是每个 不等式的右侧，则系统有一个解。我们有 这样的组合：

现在我们有了一个新的少了一个变量不等式系统。

时间复杂度

在 个不等式上消元可以最多得到 个不等式，因此连续运行 步可以得到最多 的双指数复杂度。这是由于算法产生了许多不必要的约束（其他约束隐含的约束）。必要约束的数量以单一指数增长。

可以使用线性规划 (Linear Programming, LP) 检测不必要的约束。

应用

信息论的可实现性证明保证了存在性能良好的编码方案的条件。这些条件通常使用线性不等式系统描述。系统的变量包括传输速率和附加辅助速率。通常，人们旨在仅根据问题的参数（即传输速率）来描述通信的基本限制，因此述辅助率需要消除上。而我们正是通过傅立叶 - 莫茨金消元法来做到这一点的。

实现

在编程语言中，[Racket](#)，一种基于 Lisp 的多范式编程语言在 [fme - Fourier-Motzkin Elimination for Integer Systems](#)) 中对 FME 算法做了简单函数代数实现。

参考资料与拓展阅读

1. Rui-Juan Jing, Marc Moreno-Maza, Delaram Talaashrafi, "[Complexity Estimates for Fourier-Motzkin Elimination](#)", Journal of Functional Programming 16:2 (2006) pp 197-217.
2. [Fourier-Motzkin elimination - Wikipedia](#)
3. [Fourier-Motzkin elimination and its dual](#)
4. GE Liepins,[Fourier-Motzkin elimination for mixed systems](#), 1983

本页面最近更新：2023/6/27 13:39:08, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Enter-tainer](#), [hbghlyj](#), [iamtwz](#), [isdanni](#), [sshwy](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用